MerchOps

Inventory & order-desk SaaS

Gap report against the target: multi-tenant SaaS, 100k users,
AWS production deployment, Zod + middleware security layers, best-in-class UI

At a glance

Verdict	A well-structured MVP / demo. **Not production-ready, and not yet a SaaS.** Do not expose it to the internet as-is.
Scope reviewed	182 files: Next.js 16 frontend (28 route handlers), Hono backend (6 routes), Prisma schema (9 models), Dockerfiles, compose, docs, test script.
Findings	7 critical, 10 high, 8 medium, plus 9 UI/UX gaps and a 34-endpoint security matrix (Appendix A).
Biggest risks	Unauthenticated write endpoints, forgeable session cookie, no tenant isolation, oversell race, order-number collapse at ORD-10000.
Method	Static read of every backend file and all API routes, plus the main UI shell. I did not run the app or install dependencies (no network in my sandbox). Anything about runtime behaviour is inferred from the code and marked as such.

Contents

1 Executive summary and scorecard
2 What is already good
3 Critical findings (stop-ship)
4 High-severity findings
5 Medium findings
6 Inventory-domain gaps
7 SaaS and multi-tenancy gaps
8 Scalability to 100k users
9 AWS production readiness
10 UI / UX review
11 Roadmap and exit criteria
Appendix A, B Endpoint matrix, Zod coverage

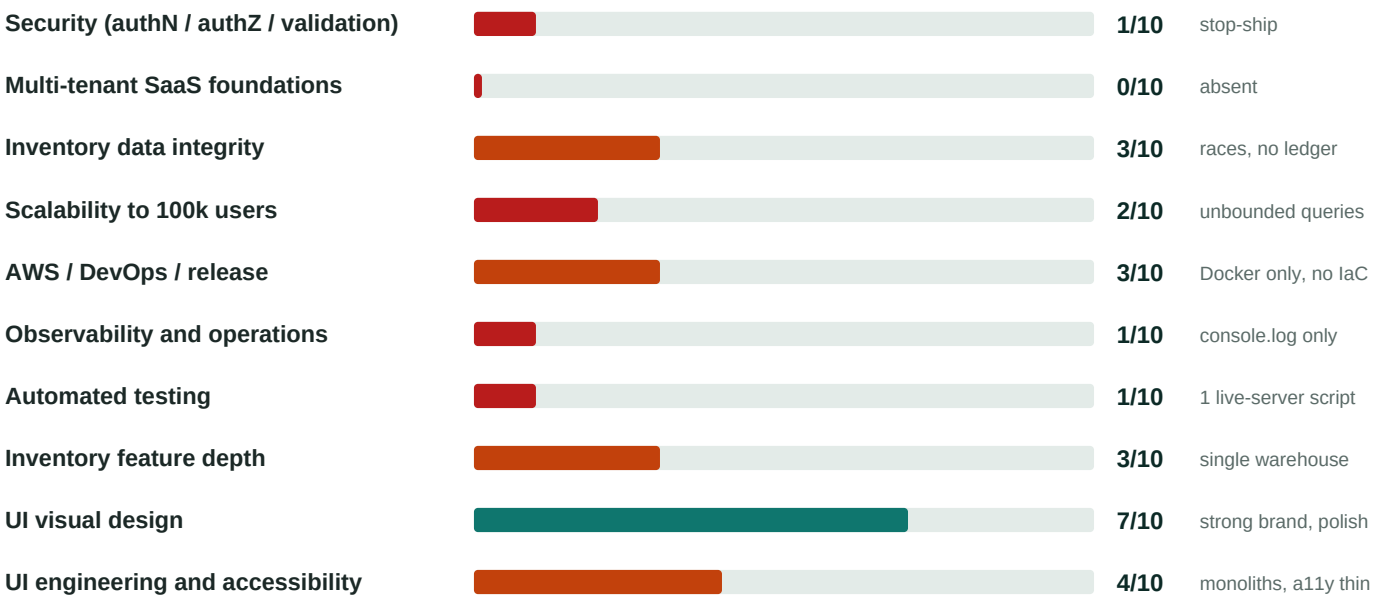
Review date: 25 October 2026 | Source: MerchOps-main.zip (static code review, 182 files)

1 Executive summary

MerchOps today is a convincing prototype of an inventory and multi-channel order desk for Bangladeshi merchants: a clean Next.js UI in English and Bangla, Prisma on PostgreSQL, a courier-adapter pattern, an audit log, and a Telegram alert. The product thinking is good. The engineering, however, is at demo level. Measured against what you asked for (a SaaS inventory system for 100k users on AWS with Zod, middleware and security layers in place, and top-tier UI), five structural gaps stand out:

- **Security is the blocker.** At least 8 endpoints that read or change data need no login, the session cookie is just a raw user ID that anyone can forge, and PINs are 4 digits with no lockout. Zod validation covers 2 of 28 Next.js handlers and 0 of 6 Hono handlers.
- **It is single-tenant.** There is no Organisation/Tenant model, no tenant ID on any table, and SKUs, order numbers, courier connections and settings are global. A SaaS needs this before any second customer signs up.
- **Inventory correctness is not guaranteed.** Concurrent orders can oversell, failed and deleted orders never restore stock, stock edits overwrite rather than adjust, and the order-number generator stops working at ORD-10000.
- **The architecture split is cosmetic.** The frontend holds all business logic and talks to the database directly. The Hono "backend" is a thin, unauthenticated side service the UI never calls, and the shared code is copied byte-for-byte into both packages.
- **Couriers are simulated.** Every adapter returns a fabricated consignment ID, and the "test connection" button marks a courier CONNECTED without calling the provider. Operators could believe a parcel was booked when it was not.

Readiness scorecard



Scores are my judgement from the static review, scaled so that 10 means "ready for a paying production customer base". They are meant to rank where effort goes, not to be precise.

Bottom line. Roughly 2 to 3 weeks of focused work fixes the stop-ship security and data-integrity items and makes a safe single-tenant pilot possible. A genuine multi-tenant, 100k-user, AWS-hosted SaaS is a 3 to 4 month programme for a team of 3 to 4 engineers (see section 11). Those are planning estimates, not commitments.

2 What is already good

A fair review should keep what works. These are real strengths to build on rather than rewrite:

- **Sound stack choices.** TypeScript, Prisma, PostgreSQL, Next.js App Router, Hono and Zod are all appropriate. Zod 4 is already installed in both packages.
- **Courier adapter pattern.** A `CourierAdapter` interface with a registry is the right shape. Swapping in real HTTP calls is additive, not a redesign.
- **Transactional order creation.** Stock decrement, order insert and audit writes share one Prisma transaction, and audit entries record previous and new values.
- **Cost redaction by role.** Factory cost and margin are nulled for non-admins on products, analytics and order detail, and product deletion is blocked when order lines exist.
- **Sensible cookie basics.** httpOnly, SameSite=Lax and Secure in production; bcrypt PIN hashes; `ALLOW_DEMO_ACCOUNTS` and `ALLOW_ROLE_SWITCH` env gates exist.
- **Local-market fit.** Asia/Dhaka day and month boundaries, BDT formatting, Bangla UI with a proper Bengali font, and a print-ready packing slip.
- **Coherent visual identity.** Deep teal and gold palette, serif display type with a clean sans body, responsive shell, toasts and an error boundary.
- **Documentation habit.** PRD, structure, system design, functional design and an API catalogue exist, which is rare at this stage.

Stack snapshot

Layer	Today	Production concern
Frontend	Next.js 16, React 19, Tailwind 3, lucide, i18n (en/bn)	All API logic lives here; DB credentials in the web container
Backend	Hono 4 on Node 22 via tsx; 6 routes	No auth, reflect-origin CORS, 3 PrismaClient instances
Data	PostgreSQL 16 + Prisma 5.22; 9 models; `db push`	No migrations, no tenant ID, no indexes, Float money
Deploy	Vercel (live) and Docker Compose	No IaC, CI, secrets management or autoscaling

3 Critical findings (stop-ship)

Each finding lists where it was observed, what it means, and the recommended fix. File paths are relative to the repo root.

CRITICAL C1 Data-changing endpoints reachable without a login

Evidence	Middleware matches only page routes (`frontend/src/middleware.ts`); `/api/*` is never gated. `PATCH /api/orders/[id]` never checks for a user, so status change, partial/exchange (which moves stock), merge and edit all work anonymously. `/api/inbox` GET, POST and PATCH have no auth. `GET /api/analytics` returns *failedFollowUps* as full order rows (names, phones) to anonymous callers. `POST /api/webhooks/order` has no secret. On the Hono side, `/api/couriers/:id/test` and `/ship` have no auth at all.
Impact	Anyone on the internet can alter orders and inventory, read customer PII, spam the inbox (each POST also fires a Telegram message) and mark parcels as shipped.
Fix	Deny by default: a `withAuth({roles, permissions})` wrapper around every handler, an auth middleware on the Hono app, and a CI test that fails if any route lacks an explicit policy. See Appendix A.

CRITICAL C2 Session cookie is the raw user ID: forgeable, non-expiring, non-revocable

Evidence	`server/auth.ts` sets cookie `merchops_user_id` to `user.id` with no signature, no maxAge and no server-side session. Middleware only checks that the cookie exists. `GET /api/orders` returns *createdBy {id, name, role}*, so a staff user can read an admin's ID from any order the admin created and set it as their own cookie.
Impact	Privilege escalation to ADMIN by any logged-in user; stolen IDs never expire; logout does not invalidate anything.
Fix	Use a vetted library (Auth.js, Lucia or iron-session) with signed or encrypted tokens, 15-minute access plus rotating refresh, a server-side session table or Redis for revocation, the `__Host-` cookie prefix, and rotation on login and privilege change.

CRITICAL C3 Weak authentication: 4-digit PINs, no lockout, demo credentials in the shipping path

Evidence	`loginSchema` allows PINs of 4 characters; there is no rate limiting or lockout, so 10,000 guesses suffice. The login form pre-fills `1234`. `GET /api/users?demo=1` returns plaintext PINs when the flag is on, and `docker-compose.yml` sets `ALLOW_DEMO_ACCOUNTS=true` under NODE_ENV=production. `verifyPin` still accepts legacy plaintext PINs. The role switcher logs in as any user ID with no PIN when `ALLOW_ROLE_SWITCH` is on, which is the non-production default. The compose command re-runs the seed on every start.
Impact	Brute-forceable admin access; demo accounts *admin/1234* survive into production data.
Fix	Passwords of 12+ characters or passkeys, argon2id, TOTP MFA for admins, Redis-backed rate limiting and progressive lockout per account and IP, email verification and reset flows, and removal of demo and role-switch code from production builds entirely (not just flags).

CRITICAL C4 No multi-tenancy: this is a single-business app, not yet a SaaS

Evidence	`schema.prisma` has no Organisation or Tenant model and no tenantId on any table. `Product.sku` and `Order.orderNumber` are globally unique, `DeliveryConnection` is `@@unique([provider])` (one Pathao account for the whole deployment), and `AppSetting` is global.
Impact	Two customers would see each other's orders, share stock and overwrite each other's courier keys. A single missed filter becomes a cross-customer data breach.
Fix	Add Organisation, Membership and per-tenant roles; put `tenantId` on every table with composite uniques (tenantId+sku, tenantId+orderNumber); derive tenant from the session, never from the request body; enforce with PostgreSQL row-level security (`SET app.tenant_id` per transaction) as a second layer; add cross-tenant tests.

CRITICAL C5 Order webhook is unauthenticated, unvalidated, and can inflate stock

Evidence	`webhooks/order/route.ts` reads *body.items* raw. In `createOrder` the check is `stockQuantity < orderedQuantity`; a negative quantity passes it and the `decrement` of a negative number *increases* stock. A caller-supplied *unitPrice* is accepted, so negative or zero totals are possible. Errors echo `error.message` to the caller.
Impact	Unauthenticated stock manipulation, fake revenue, and a free trigger for Telegram notifications. There is no idempotency, so a retried webhook creates a duplicate order and a second stock decrement.
Fix	Per-tenant, per-channel HMAC signature with a timestamp window, a Zod schema (positive integer quantities, price from the catalogue only), a unique constraint on (tenantId, channel, externalId) for idempotency, and processing through a queue so the sender gets a fast 202.

CRITICAL C6 Inventory integrity: oversell race, stock never restored, order-number collapse

Evidence	**Race:** `createOrder` reads products *outside* the transaction and checks the stale snapshot, then decrements; two concurrent orders for the last unit both pass and stock goes negative (no CHECK constraint). **No restock:** `updateOrderStatus` to FAILED and `DELETE /api/orders/[id]` never return reserved stock; only partial/exchange returns restock. **Overwrite:** `PATCH /api/products` sets an absolute *stockQuantity*, losing concurrent changes. **Order numbers:** `nextOrderNumber` orders by the *string orderNumber desc*. Starting at 1001, "ORD-9999" sorts above "ORD-10000", so after ~9,000 orders it keeps proposing ORD-10000 and every new order hits the unique constraint. Even before that, concurrent creates compute the same number. **Bulk entry** runs up to 50 separate transactions, so a failure leaves a partial batch.
Impact	Overselling, silent inventory drift, and a hard stop on order creation at roughly 9,000 orders (days, at 100k-user volumes).
Fix	Atomic conditional update (`UPDATE ... SET qty = qty - $n WHERE qty >= $n`, check rows affected) plus a CHECK (qty >= 0); an append-only `StockMovement` ledger as the source of truth; a per-tenant Postgres sequence for order numbers; an explicit order state machine that restocks on FAILED/cancel; adjustments as deltas with a reason code; one transaction per bulk batch.

CRITICAL C7 Courier integrations are simulated, but the UI presents them as live

Evidence	`couriers/base.ts` `demoConsignment()` fabricates IDs such as *PATHAO-ORD-1001-lx3k*; no adapter makes an HTTP call. `testConnection` returns ok if a key is non-empty. The Next route's *test* action sleeps 600 ms and sets status CONNECTED without contacting the provider. Shipping then moves the order to IN_PROGRESS.
Impact	A merchant could dispatch real customers' orders believing a courier booking exists. No tracking, label, COD reconciliation or delivery-status sync exists either.
Fix	Implement each adapter against the provider API (OAuth token refresh for Pathao, etc.) with timeouts, retries with backoff, circuit breakers and idempotency keys; run calls in a worker; ingest courier status webhooks; label sandbox mode clearly in the UI until real calls ship.

4 High-severity findings

HIGH **H1** **Secrets stored and leaked in plaintext**

Evidence	Courier *apiKey* and *apiSecret* are plain columns. `GET /api/settings` returns the Telegram bot token. In `PUT /api/settings` the audit entry redacts *newValue* but writes the whole *previous* row, token included, into `AuditLog`. Admin-configurable *apiBaseUrl* will become an SSRF vector (for example to the EC2 metadata address) once adapters make real calls.
Impact	A database read or backup leak exposes every merchant's courier account; the audit table becomes a secrets store.
Fix	Envelope-encrypt secrets with KMS (or keep them in Secrets Manager), never return them from APIs, redact in audit, rotate on change, and allow-list courier base URLs per provider with egress filtering.

HIGH **H2** **Role checks are inconsistent; Guest (read-only per the PRD) can modify data**

Evidence	`POST /api/orders` and `POST .../ship` require only a session, so Guest can create orders and ship. `PATCH /api/products` lets any session change stock, name and threshold. Status, partial and merge actions in `PATCH /api/orders/[id]` have no role check at all; only *edit* blocks GUEST, and it does so with `user?.role`, which an anonymous caller passes.
Impact	The three-role model is advisory rather than enforced.
Fix	One central permission map (resource x action x role) enforced in a policy layer (for example CASL), with a generated test that walks every route for every role.

HIGH **H3** **Input validation almost absent; money stored as Float**

Evidence	Zod runs on login and order creation only. `productPatchSchema` exists but is never used. Everything else uses raw `req.json()` and `Number(...)` casts: *NaN* reaches Prisma, query params are cast with `as DeliveryStatus`, `new Date(from)` can be Invalid Date, *limit* can be NaN. The CSV export escapes quotes but not leading = + - @, enabling spreadsheet formula injection from webhook-supplied names. Prices and totals are `Float`.
Impact	500 errors on bad input, injection into exported files, and rounding drift in financial totals.
Fix	A shared `@merchops/contracts` package of Zod schemas used for request parsing, response typing and OpenAPI generation; reject unknown keys with `.strict()`; `Decimal(12,2)` or integer minor units for money; prefix-escape CSV cells that start with formula characters. Coverage table in Appendix B.

HIGH **H4** **No rate limiting, security headers, CSRF strategy or body limits; CORS reflects any origin**

Evidence	No limiter anywhere. `next.config.ts` defines no headers (no CSP, HSTS, frame-ancestors, Referrer-Policy). Hono CORS is `origin: (o) => o || "*"` with `credentials: true`, which lets any website make credentialed cross-origin calls once cookies are accepted. No request size caps; no request IDs.
Impact	Credential stuffing, scraping, clickjacking, and cross-site abuse of the API.
Fix	WAF plus Redis token-bucket limits (login, webhooks, exports), `secureHeaders` and a strict CSP, an explicit CORS allow-list, double-submit or Origin-checked CSRF tokens for cookie auth, 1 MB body limits, and a request-ID middleware feeding logs.

HIGH **H5** **Unbounded queries and zero indexes: breaks long before 100k users**

Evidence	Orders, products and inbox GET endpoints, and the CSV export, return every row with nested items and products; none paginate. `getDashboardMetrics` loads *all products* and *every order of the month with items* into memory on each call and aggregates in JavaScript. Order search uses `contains` on five columns (case-sensitive, full scans). The schema has no `@@index` at all, and Postgres does not index foreign keys automatically.
Impact	Latency and memory grow linearly with data; one large tenant can stall the API for everyone.
Fix	Cursor pagination everywhere (max 100), indexes such as (tenantId, createdAt desc), (tenantId, deliveryStatus), OrderItem(orderId), OrderItem(productId), AuditLog(tenantId, timestamp), pg_trgm GIN indexes for search, SQL aggregates (GROUP BY) instead of JS loops, and a materialised or cached dashboard refreshed by a worker.

HIGH **H6** **Database connection handling and health check**

Evidence	The backend creates three separate `new PrismaClient()` instances (`routes/health.ts`, `routes/couriers.ts` and `lib/prisma.ts`), each with its own pool. `/api/health` runs three `COUNT(*)` queries per call and publishes user, order and product counts.
Impact	Connection exhaustion under autoscaling, and a load balancer health check that scans the largest tables every few seconds while leaking business metrics.
Fix	One shared client, `connection_limit` tuned per task, RDS Proxy or PgBouncer in front of Aurora, a `/healthz` that does `SELECT 1` only, and a separate authenticated status page for counts.

HIGH **H7** **The frontend/backend split is cosmetic and the code is duplicated**

Evidence	Nothing in `frontend/src` references `NEXT_PUBLIC_API_URL`; all UI calls go to Next route handlers that use Prisma directly, so the frontend container needs `DATABASE_URL`. `backend/src/lib` is a byte-identical copy of `frontend/src/lib` (a directory diff shows only *auth.ts* and *config.ts* exist on one side), including a React `i18n/provider.tsx`, and `backend/src/lib/auth.ts` imports `next/headers` although the backend has no Next dependency.
Impact	Two deployables with different behaviour, fixes that land in only one copy, and no network boundary protecting the database from the internet-facing tier.
Fix	Pick one model. Recommended: a single API service (Hono, Fastify or NestJS) that is the only process allowed to reach the database, plus a Next.js app that renders UI and calls it. Share types and Zod schemas through a workspace package instead of copying files.

HIGH **H8** **Build, container and release pipeline is not production-grade**

Evidence	Backend Dockerfile: `tsc ... || true` hides compile errors, runtime runs `tsx` from source, and the root lockfile is not in the build context. By inspection `tsc` will fail: NodeNext resolution needs *.js* extensions on relative imports, which the code omits, and lib files import Next and JSX. Frontend Dockerfile: `npm install || true`, runs as root, no HEALTHCHECK, no *standalone* output, and `next.config.ts` is not copied into the runner. Compose runs `prisma db push` and the seed at every start, publishes Postgres on 5432 with the password *merchops*, and there is no `prisma/migrations` folder. No `.github` workflows.
Impact	Broken builds ship silently, schema changes can drop data, and no release can be reproduced or rolled back.
Fix	Remove every `|| true`; multi-stage builds with `npm ci`, a non-root user, `output: "standalone"`, HEALTHCHECK and graceful SIGTERM; `prisma migrate deploy` as a one-off release task; GitHub Actions for lint, typecheck, test, image scan and deploy.

HIGH **H9** **No automated tests**

Evidence	One `scripts/test-features.mjs` with 26 assertions that needs a running dev server and seeded demo data. `npm run lint` is only `tsc --noEmit`; the ESLint config is not wired to a script.
Impact	Every change risks silent regressions in stock maths, permissions and money.
Fix	Vitest unit tests for domain logic, Testcontainers-based integration tests per route and role, a concurrency test (200 parallel orders on one SKU must never oversell), Playwright end-to-end for core flows, and k6 load tests in CI.

HIGH **H10** **Observability and operability gaps**

Evidence	Logging is `console.log` and Hono's default logger; no structured logs, error tracking, metrics or traces. `index.ts` has no SIGTERM handler, so ECS deploys drop in-flight requests. Telegram sends are awaited inside user requests with no timeout, after an extra settings query on each call. No backup, restore or retention policy is documented.
Impact	Incidents will be diagnosed from user reports; deploys cause errors; a slow Telegram API slows order entry.
Fix	Pino JSON logs with request and tenant IDs, OpenTelemetry to CloudWatch/X-Ray, Sentry, RED metrics and alarms (5xx, p95, queue age, DB CPU/connections), graceful shutdown, notifications moved to a queue, and documented RPO/RTO with a tested restore.

5 Medium findings

ID	Finding	Detail	Fix
M1	**Accounting accuracy**	Revenue and profit are bucketed by order *createdAt*, not delivery date. COGS uses the product's *current* factoryCost, not the cost at sale time. Ad-profit revenue uses catalogue *retailPrice* instead of actual unit price. *costPrice* and *factoryCost* overlap with no defined meaning.	Store cost snapshots on OrderItem; recognise revenue on delivery; define one cost model; add FIFO or weighted-average valuation.
M2	**Audit log is not tamper-resistant**	Called "immutable" in docs but it is an ordinary table with no DB protections; entries hold full JSON of products and orders including customer PII; no retention or partitioning; the audit API casts *action as never*.	Revoke UPDATE/DELETE for the app role, hash-chain or ship to S3 Object Lock, partition by month, redact PII, add retention.
M3	**No order state machine; hard deletes**	Any status can move to any other (DELIVERED back to PENDING). Orders are hard-deleted; merge marks the secondary order FAILED, which pollutes failure metrics.	Explicit transitions with guards and side effects; soft-delete with deletedAt; a distinct MERGED/CANCELLED status.
M4	**Schema typing left over from SQLite**	Enums, tags and settings are strings or JSON-in-string columns (*tags*, *configJson*); schema comments still describe SQLite as the default.	Native Postgres enums, *jsonb* or a Tag table, Decimal money, and updated comments. Migrate while the data set is small.
M5	**Locale, currency and timezone hard-coded**	Asia/Dhaka and BDT are baked into helpers and the AdSpend default, which blocks customers elsewhere.	Per-tenant timezone, currency and locale settings; keep Bangla as a first-class locale.
M6	**Notifications limited to Telegram**	No email, in-app, SMS or WhatsApp; no per-user preferences or digest; failures are swallowed.	Notification service with channels, templates, preferences and retries via the queue.
M7	**Docs contradict each other**	GO_LIVE.md says SQLite is the default and tells users to flip the provider; REPORT.md says Postgres only; README publishes demo PINs.	Single source of truth, an ADR log, and a deployment runbook.
M8	**Public product surface incomplete for SaaS**	No terms, privacy policy, pricing, status page, security page, onboarding wizard, or data export/delete (GDPR-style) flow.	Add marketing and legal pages and tenant self-service data export.

6 Inventory-domain gaps

Compared with a credible inventory-management SaaS, the current model covers products, stock counts and orders, but little of the surrounding workflow:

Area	Today	Expected for a competitive product
Stock model	Single quantity per product	On-hand vs reserved vs available; stock movement ledger; per-location balances
Locations	None	Multiple warehouses/bins, transfers between locations
Replenishment	Static lowStockThreshold flag	Reorder points and quantities, suppliers, purchase orders, goods receipt, lead times, auto-suggested POs
Catalogue	Flat product; variant encoded in SKU (TEE-BLK-M)	Variants/options, categories, units of measure, images, barcodes (EAN/UPC), bundles
Traceability	None	Batch/lot and expiry, serial numbers where relevant
Counting	Manual absolute edit	Stocktake / cycle counts with variance reports and approval
Valuation	Current cost x quantity	FIFO / weighted average, cost layers, landed cost

Area	Today	Expected for a competitive product
Returns	Partial/exchange adjusts stock	RMA workflow with inspection and restock/scrap decision
Channels	Webhook + manual inbox	Official Shopify, WooCommerce, Meta Cloud API / WhatsApp Business connectors with two-way stock sync and oversell protection
Import / export	Orders CSV export only	Bulk product and stock import with validation preview; scheduled exports
Public API	None	Per-tenant API keys, rate-limited REST, outbound webhooks, OpenAPI docs
Reporting	Monthly P&L; on dashboard	Sell-through, ABC analysis, ageing, stock-out history, courier performance, COD reconciliation

7 SaaS and multi-tenancy gaps

Beyond tenant isolation (C4), a subscription product needs a layer the repo does not have yet:

Capability	What is needed
Identity	Email + password or passkey, verification, reset, invitations, MFA, SSO (Google/Microsoft; SAML for enterprise), session management page, per-tenant roles and custom roles
Tenancy	Organisation lifecycle (create, suspend, delete), tenant-scoped everything, per-tenant config (currency, timezone, numbering), tenant-aware caching keys and queue messages
Billing	Plans and entitlements (users, SKUs, orders/month, channels), Stripe or a Bangladesh-capable gateway (SSLCommerz, bKash) for payments, trials, dunning, invoices, usage metering
Admin plane	Internal back-office to support tenants, impersonate with audit, feature flags, and plan overrides
Compliance	Privacy policy and terms, data processing terms, data export and erasure, audit export, backup/restore policy, penetration test, and an incident process
Customer-facing reliability	Public status page, SLA targets, maintenance windows, in-app announcements, support tooling

8 Scalability to 100k users

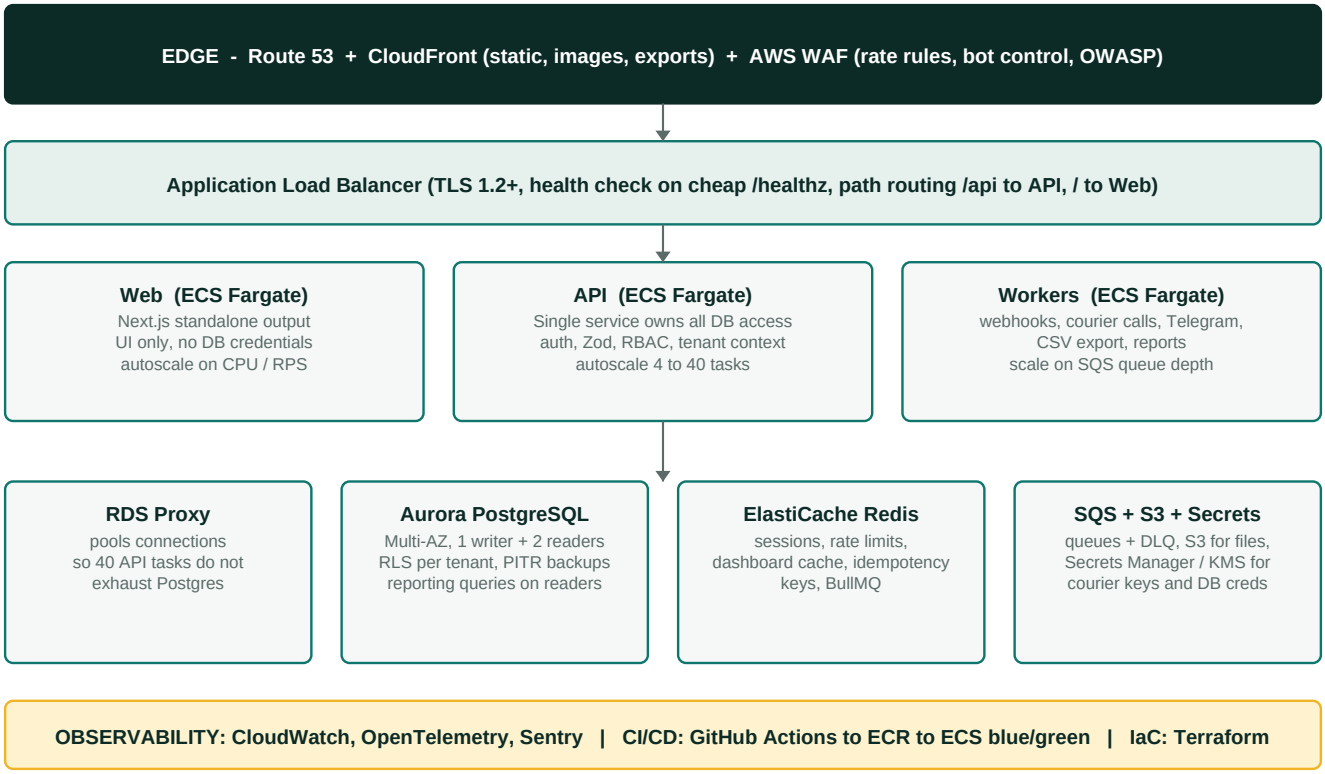
"100k users" can mean very different loads, so I sized against an explicit planning assumption you should replace with real telemetry:

> **Planning assumption (illustrative).** 100,000 registered users; about 10% active at peak = 10,000 concurrent sessions; one API call per ~10 s per active user = roughly **1,000 requests/s peak**, 90% reads. Add webhook bursts (flash sales) of several hundred orders/min per large tenant.

Where the current design breaks

Area	Today	Target
Order creation	Hard stop at ORD-10000; oversell race; stale reads outside tx	Per-tenant sequence; atomic conditional UPDATE; ledger
List pages	Return all rows with joins; no pagination	Cursor pagination, covering indexes, virtualised tables
Dashboard	Loads full month of orders into Node memory per request	SQL aggregates, Redis cache (30-60 s), pre-computed daily rollups by a worker
Search	contains on 5 columns, no indexes	pg_trgm GIN or OpenSearch for full-text
Auth	bcryptjs (pure JS) at cost 10 blocks the event loop; no limiter	argon2id/native bcrypt in a worker pool; Redis limiter; session cache
Webhooks	Synchronous: DB writes + Telegram call in the request	202 Accepted, SQS queue, idempotent worker, DLQ
Couriers	(Future) provider calls inline with no timeout	Worker with timeouts, retries, circuit breaker, per-provider concurrency caps
DB connections	3 Prisma pools per backend task; web tier also connects	Single API tier, RDS Proxy, tuned pool, read replicas for reports
Static assets	1 MB and 800 KB JPEGs used as CSS backgrounds	CDN, next/image with AVIF/WebP and responsive sizes

Reference target architecture on AWS



Service names are a recommendation, not the only valid choice; the principles are the point: a single database-owning API tier, stateless containers, connection pooling, a queue for anything slow or external, a cache for sessions and hot reads, and tenant isolation enforced at the application *and* database layers.

9 AWS production readiness

Area	Today	Required
Infrastructure as code	None	Terraform or CDK for VPC (private subnets), ALB, ECS, Aurora, Redis, SQS, S3, IAM, WAF; separate dev/staging/prod accounts
Networking and access	Compose publishes 5432; DB reachable from host	DB in private subnets, security groups API-only, no public DB endpoint, VPC endpoints for S3/SQS/Secrets
Secrets	Env vars and plaintext DB columns	Secrets Manager + KMS; task-role access; rotation; no secrets in images or repos
Database operations	db push at container start; seed on start	Versioned migrations run as a one-off task before deploy; expand-and-contract for zero downtime; PITR and cross-region snapshots
CI/CD	None; Vercel auto-build only	GitHub Actions: typecheck, lint, tests, SCA/SAST, image scan (ECR/Trivy), then blue/green with automatic rollback
Autoscaling	Single container each	ECS service autoscaling on CPU/RPS; worker scaling on SQS depth; Aurora read-replica scaling; load-tested limits
Resilience	No timeouts, retries or graceful stop	Multi-AZ, health checks, SIGTERM handling, circuit breakers, DLQs, chaos/restore drills
Security services	None	WAF managed rules + rate rules, GuardDuty, CloudTrail, Security Hub, Inspector, least-privilege IAM, VPC flow logs
Cost and capacity	Unknown	Budgets and alarms, rightsizing from load tests, Savings Plans once baseline is known

Environment hygiene checklist

- Set `ALLOW_DEMO_ACCOUNTS=false` and `ALLOW_ROLE_SWITCH=false` in every deployed environment, then remove the code paths.
- Validate all environment variables at boot with a Zod schema and fail fast on missing or default secrets.
- Keep the Vercel deployment and the AWS deployment from sharing a database; today both point at the same Postgres design and the same demo seed.
- Rotate any credential that has appeared in `REPORT.md`, `README.md` or the live demo.

10 UI / UX review

Strengths. A confident, consistent identity (deep teal sidebar, gold accents, serif display headings, Manrope body), a responsive shell, bilingual text with a proper Bengali face, toasts, an error boundary, and a print stylesheet for packing slips. For a "best quality UI" target the gaps below are about depth, accessibility and engineering rather than looks.

ID	Gap	Observation	Recommendation
U1	**Monolithic screens, no data layer**	`orders-desk.tsx` is 1,065 lines with 7 raw *fetch* calls; only 2 of 17 components reference any loading state. No caching, optimistic updates, retries or skeletons.	Adopt TanStack Query, split into table / filters / drawers / forms, add skeletons and optimistic updates.
U2	**Tables will not scale**	Inline-edit tables render every row; no pagination, virtualisation, column sorting from the server, saved views or URL-synced filters.	Server-side pagination, TanStack Table + virtualisation, saved filters, keyboard navigation.
U3	**Accessibility**	About 11 aria/role/alt attributes across all components; icon-only buttons rely on *title*; inline-edit grids have no keyboard or screen-reader model; no contrast audit (for example small muted text on the dark sidebar).	WCAG 2.2 AA audit with axe in CI, labelled controls, focus rings, roving tabindex in grids, reduced-motion support.

ID	Gap	Observation	Recommendation
U4	**Mobile and warehouse use**	On small screens navigation is a horizontally scrolling strip; GO_LIVE lists a bottom nav as a check item but the shell does not provide one. No camera barcode scan, large-target pick/pack mode or offline tolerance.	Bottom tab bar, PWA with offline queue, barcode scanning, pack-station mode.
U5	**Asset performance**	Two large JPEGs (about 1.9 MB combined) served as CSS backgrounds, the hero reused on four surfaces, three Google font families, no *next/image*.	Responsive AVIF/WebP via next/image, preload only above-the-fold, subset fonts, Lighthouse budget in CI.
U6	**Thin design system**	`components/ui` holds only badge, button, input and textarea; button styles are duplicated as global *@apply* classes.	Radix/shadcn primitives (dialog, select, combobox, date picker, dropdown, tabs, tooltip), documented tokens, Storybook, optional dark mode and density toggle.
U7	**Demo artefacts in the product**	Role switcher in the header, quick-login chips and a pre-filled PIN on the login form.	Strip from production builds; replace with a user menu showing tenant, role and sign-out.
U8	**Missing inventory UX**	No stock-adjustment dialog with reason, movement history, bulk actions, global search or command palette, CSV import wizard, notification centre, or product images.	Build the flows listed in section 6 with undo and confirmation patterns.
U9	**SaaS front door**	Landing page lacks pricing, security/trust, legal pages, onboarding checklist and meaningful empty states.	Marketing site, guided onboarding with sample data, contextual help.

11 Prioritised roadmap and exit criteria

Effort figures are rough planning estimates for a team of 3 to 4 engineers and should be re-estimated after a short spike on phases 1 and 2.

Phase	Closes	Scope	Exit criteria
Phase 0 Week 0-1 Stop-ship	C1, C2, C3, C5, H1, H4 (basics)	Auth wrapper on all routes; signed sessions; remove demo and role-switch from prod; webhook HMAC + Zod; rate limits on login and webhooks; rotate exposed secrets; security headers; fix CORS.	No anonymous access to any data route (CI-enforced); login lockout works; pen-test smoke passes.
Phase 1 Weeks 2-5 Foundations	C4, C6, H3, H5, H6, H7, H8, H9	Tenant model + RLS; migrations; StockMovement ledger and atomic stock updates; order sequences and state machine; Zod contracts package; single API service; indexes and pagination; CI pipeline; first integration tests.	200 concurrent orders on one SKU never oversell; cross-tenant tests green; migrations run in CI.
Phase 2 Weeks 6-10 Cloud + scale	C7, H10, M2, M6	Terraform AWS stack; ECS services; RDS Proxy; Redis; SQS workers; real Pathao/Steadfast adapters with status webhooks; observability and alarms; secrets in Secrets Manager/KMS.	Blue/green deploy with rollback; restore drill passes; real courier booking in sandbox.
Phase 3 Weeks 10-14 Product + polish	U1-U9, section 6 and 7 features	Design system and accessibility pass; table virtualisation; barcode/PWA mode; billing and plans; onboarding; suppliers and POs; reports.	WCAG 2.2 AA audit clean; Lighthouse performance 90+ on key pages; paid plan end to end.
Phase 4 Weeks 14-16 Prove it	Load and security validation	k6 tests to 1,000+ req/s with mixed read/write; soak test; external penetration test; DR and failover game day; capacity and cost review.	Targets below met with headroom; high/critical pen-test findings closed.

Production-readiness targets (definition of done)

Dimension	Target
Availability	99.9% monthly for the API; zero-downtime deploys
Latency (at ~1,000 req/s)	p95 under 300 ms for reads and under 600 ms for writes; dashboard under 500 ms warm
Correctness	No negative stock under concurrency; every stock change has a ledger row; money exact to the minor unit
Security	Deny-by-default routes; 100% Zod coverage on inputs; MFA for admins; secrets encrypted; no high/critical pen-test findings
Recovery	RPO 5 minutes or better (PITR); RTO 30 minutes or better; restore tested quarterly
Quality gates	CI blocks on typecheck, lint, unit/integration tests, dependency and image scans; coverage on domain logic above 80%
Accessibility and speed	WCAG 2.2 AA on core flows; LCP under 2.5 s on 4G for the landing and login pages

Suggested first three tickets tomorrow: (1) add a deny-by-default `withAuth` wrapper and apply it to all 34 handlers; (2) replace the user-ID cookie with a signed, expiring session and delete the role switcher and demo endpoint; (3) rewrite `createOrder` to use an atomic conditional stock update and a database sequence for order numbers.

Appendix A Endpoint security matrix

Every route handler found in the repo. **Auth** shows what the code actually enforces. Red rows (8) are reachable without a session and read or change data; the other NONE rows crash or are harmless.

Verb	Route	Auth enforced	Zod	Notes
POST	`/api/session`	Public (by design)	Yes	No limiter; userId-only role switch when flag on
GET	`/api/session`	Cookie	n/a	Trusts raw user-ID cookie
DELETE	`/api/session`	None	n/a	Does not invalidate a session
GET	`/api/users`	Session; ?demo=1 public	No	Demo mode exposes plaintext PINs
GET	`/api/orders`	Any session	No	Unbounded; leaks createdBy id/role
POST	`/api/orders`	Any session incl. Guest	Yes	Bulk not atomic
GET	`/api/orders/[id]`	**NONE**	No	Crashes (500) for anon via user!.role
PATCH	`/api/orders/[id]`	**NONE**	No	status / partial / merge / edit unauthenticated
DELETE	`/api/orders/[id]`	Non-guest session	No	Hard delete, no restock
GET	`/api/orders/export`	Any session	No	Unbounded; CSV formula injection
GET	`/api/products`	Any session	No	Unbounded
POST	`/api/products`	Admin	No	Raw Number() casts
PATCH	`/api/products`	Any session incl. Guest	No	Stock/name edits open to Guest; schema unused
DELETE	`/api/products`	Admin	No	
GET	`/api/inbox`	**NONE**	No	Customer messages exposed
POST	`/api/inbox`	**NONE**	No	Spam + Telegram trigger
PATCH	`/api/inbox`	**NONE**	No	Anyone can relink/close messages
GET	`/api/analytics`	**NONE**	No	Returns failed-order PII to anon
POST	`/api/analytics`	Admin	No	
GET	`/api/audit`	Admin	No	limit NaN not handled
GET	`/api/settings`	Admin	No	Returns Telegram bot token
PUT	`/api/settings`	Admin	No	Audit stores previous token
GET	`/api/delivery-connections`	Any session	n/a	Staff view is filtered; fine
POST	`/api/delivery-connections`	Admin	No	Plaintext key storage
PATCH	`/api/delivery-connections/[id]`	Admin	No	test action fakes success
POST	`/api/delivery-connections/[id]/ship`	Any session incl. Guest	No	Fake consignment
POST	`/api/webhooks/order`	**NONE**	No	No secret; negative qty adds stock
GET	`/api/health`	Public	n/a	3x COUNT(*), exposes counts + flags
GET	`[hono] /api`	Public	n/a	
GET	`[hono] /api/health`	Public	n/a	Extra PrismaClient
GET	`[hono] /api/meta/api-catalog`	Public	n/a	Full endpoint catalogue
GET	`[hono] /api/couriers/adapters`	**NONE**	n/a	Low risk
POST	`[hono] /api/couriers/:id/test`	**NONE**	No	Changes connection status
POST	`[hono] /api/couriers/:id/ship`	**NONE**	No	Marks any order IN_PROGRESS

Appendix B Zod coverage

Measure	Result
Next.js route handlers using Zod	2 of 28 (login, order create/bulk)
Hono route handlers using Zod	0 of 6
Schemas defined but unused	`productPatchSchema`, `sourceChannelSchema` and `deliveryStatusSchema` are only referenced inside validation.ts
Unvalidated inputs	Webhook body, order PATCH (5 actions), product POST/PATCH, inbox POST/PATCH, settings PUT, courier POST/PATCH/ship, analytics POST, all query strings (status, from, to, limit, q), Hono ship body
Response validation	None; responses spread raw Prisma rows, which is how internal fields (for example createdBy details) leak
Env validation	None; flags read ad hoc from `process.env`

Recommended pattern: one `validate({ params, query, body, response })` helper that wraps each handler, reads schemas from the shared contracts package, returns a uniform RFC 7807 problem response, and also generates OpenAPI so the UI client is typed end to end.